

iteratr

.online

The Adaptive AI Coding Mentor

**Product Plan & Feature Document |
v1.0**

Pair-program with a senior engineer. Never just be told you're wrong.

Document type:

Product Plan & Feature Spec

Product:

iteratr — iteratr.online

Status:

Pre-build | Confidential

Audience:

Founder, Dev Team, Investors

1. Product Vision

The Elevator Pitch

iteratr is an autonomous, full-stack coding mentor that replicates the experience of pair-programming with a senior engineer. It dynamically scales problem difficulty using an Elo rating system, securely executes user code to analyze logic failures, and uses real-time multi-agent panels to conduct mock technical interviews — all while never just telling the user they are wrong.

1.1 The Core Problem

Every major coding platform today — LeetCode, HackerRank, Codewars — has the same fundamental flaw: they tell you if you are right or wrong, then show you the answer. This creates passive learning. The user reads the solution, thinks they understand it, then fails the same type of question two days later.

iteratr fixes this with three principles:

- Never give the answer directly — always guide with Socratic hints
- Personalize the difficulty dynamically — not static Easy/Medium/Hard tracks
- Teach through doing and failing — real code execution with real feedback

1.2 Target Users

User Segment	Description
CS students	College students preparing for internship and placement interviews. Primary growth segment.
Job seekers	Developers actively applying at FAANG, startups, or product companies. High urgency, high engagement.
Self-taught devs	Bootcamp graduates or autodidacts who need structured DSA practice without a mentor.
Working engineers	Mid-level engineers preparing for senior role interviews or switching companies.
Interview coaches	Instructors who want to assign iteratr sessions to their students and track performance.

1.3 Product Name & Brand

Name: iteratr

Domain: iteratr.online

Tagline: Pair-program with a senior engineer. Never just be told you're wrong.

Brand meaning: Iteration is the core loop of coding: write, fail, understand why, improve. The dropped 'e' signals a developer-built, developer-first product — the same naming convention used by Flickr, Tumblr, and Fiverr.

2. Feature Overview

iteratr is built in four phases. Each phase adds a layer of intelligence on top of the last. The MVP (Phases 1 and 2) is fully functional and valuable on its own. Phases 3 and 4 unlock the features that make the platform industry-relevant and defensible.

Phase	Name	Timeline	What it unlocks
Phase 1	Core Testing Engine	Weeks 1–4	Multi-type questions, code execution, hint engine
Phase 2	Adaptive Intelligence	Weeks 5–8	Elo rating, knowledge gap detection, session config
Phase 3	Mock Interview Suite	Weeks 9–13	AI interview panel, scorecards, company mode
Phase 4	Analytics & Growth	Weeks 14–16	Heatmaps, streaks, shareable profiles

2.1 Unique Differentiators

These are the features that no existing platform offers in combination. They are the reason a user chooses iteratr over LeetCode.

- Tiered Socratic hint engine — 4 levels from concept nudge to pseudocode scaffold, never the answer
- Elo rating per user AND per question — dynamic difficulty that adjusts in real time
- Knowledge gap detector — AI flags weak sub-topics and actively routes easier foundational questions
- Custom prompt injection — user writes "Act like a strict Google interviewer" and the AI obeys
- Silent grader agent — runs invisibly during mock interviews, scoring communication and code efficiency
- Company mode — FAANG, startup, and product company interview styles built in
- Shareable public profile — Elo rating, topic mastery, and interview scores. LinkedIn-ready.

3. Phase 1 — Core Testing Engine

Goal: Build the minimum viable product. A user should be able to log in, take a multi-type test, get hints when wrong, and see their result.

3.1 Multi-Type Question Engine

The question engine is the heart of iteratr. Every question is generated by the AI into a strict JSON schema. There are four question types, each testing a different cognitive layer.

Type 1 — Multiple Choice (MCQ)

Tests conceptual understanding with four options. The wrong answers (distractors) are generated from real misconceptions, not random noise. The AI is explicitly prompted to ask: "What would a student who half-understands this concept think?" This makes MCQs genuinely educational.

- 4 options: 1 correct, 1 clearly wrong, 2 plausible distractors
- Distractors map to known misconceptions (e.g. $O(\log n)$ for hash maps because students confuse them with BSTs)
- On wrong answer: show which option was correct AND explain why the chosen option fails
- Hint: never say "wrong" — say "here is why that reasoning breaks down"
- Elo update: +12 for correct first try, +6 for correct second try, -8 for fail

Type 2 — Fill in the Blank

Tests the blank that matters — not syntax trivia, but the conceptual term or value that reveals whether the student truly understands. The AI chooses the blank strategically: it picks the word whose absence forces active recall of the core concept.

- 1 to 3 blanks per question depending on complexity
- Blank is always the conceptual gap, not a random word
- Partial credit: each blank scored independently
- Hint: if wrong, describe what the blank represents without giving the answer
- Example: "A recursive function must have a ____ case to stop and a ____ case to call itself"

Type 3 — Code Space

The most powerful question type. The user writes real code in a Monaco Editor (VS Code engine). The code is sent to a secure sandboxed execution environment (Judge0 API) and run against hidden test cases. The AI agent then reasons about the failure and generates a personalized hint.

- Languages: Python, C++, JavaScript, TypeScript (Phase 1 starts with Python only)
- Partial scaffolding provided: function signature + comments, never a blank screen
- Code runs against 5 to 10 hidden test cases including edge cases
- On failure: agent identifies which test case failed and WHY (empty array, negative input, etc.)
- Hint system: 4 progressive levels (see Section 5 for full hint engine spec)
- Elo factors in: correctness, time taken, and number of hints used

Type 4 — Drag to Order

Tests procedural understanding — knowing the correct sequence of steps. Perfect for algorithms (binary search, merge sort), system design flows (what happens when you press Enter in a browser), or CS concepts (TCP handshake, OS boot sequence). Drag and drop, no typing required.

- 4 to 8 steps presented in shuffled order
- User drags to arrange in correct sequence
- On wrong order: show first incorrect step and hint about what must come before it
- Great warm-up question before a harder code space challenge on the same topic

3.2 Live Code Execution (Sandbox)

User code is NEVER executed directly on the iteratr server. All code runs in a secure, isolated Docker container via the Judge0 API or Piston. This protects the server from malicious code while providing real execution results.

Component	Implementation detail
Execution service	Judge0 API (via RapidAPI) in Phase 1. Self-hosted Piston in Phase 3+ for cost control.
Supported languages	Python 3.11 (Phase 1), C++ 17, JavaScript Node 20 (Phase 2)
Timeout	5 seconds per execution. Infinite loops are killed automatically.
Hidden test cases	5 to 10 per question. Stored server-side, never exposed to the client.
Output parsed	stdout, stderr, exit code, execution time, memory used
Edge cases tested	Empty input, null, negative numbers, max integer, single element arrays

3.3 Interactive Code Editor

The editor is built on Monaco Editor — the same engine that powers VS Code. This gives users a familiar, professional coding environment with syntax highlighting, autocomplete, and error underlining.

- Syntax highlighting for all supported languages
- Auto-indentation and bracket matching
- Line numbers and fold regions
- Read-only scaffolding section (function signature) with editable body
- Font size controls and theme toggle (light/dark)
- No copy-paste restriction — this is not a test, it is a learning tool

3.4 Authentication & User Profiles

Standard OAuth login via Google and GitHub. No password required. User profiles are created on first login.

- OAuth: Google and GitHub
- Profile fields: display name, avatar, join date, preferred language
- Profile displays: current Elo rating, total questions attempted, questions solved, streak count
- Activity heatmap: GitHub-style grid showing daily activity
- Topic mastery map: visual breakdown of strong vs weak topics

4. Phase 2 — Adaptive Intelligence

Goal: Make the platform feel intelligent. Difficulty adjusts to the user. The AI knows what they struggle with.

4.1 Elo Rating System

Every user has a hidden Elo rating (starting at 1200). Every question in the database also has an Elo rating. The system works exactly like chess matchmaking: solve a hard question quickly and your rating spikes; fail an easy one and it drops. The next question served always matches the user's current rating.

Outcome	Elo change
Correct, 0 hints, fast	+18 to +24 (depends on question Elo vs user Elo)
Correct, 1 hint used	+10 to +14
Correct, 2+ hints used	+4 to +8
Incorrect after 4 hints	-6 to -12
Time limit exceeded	-4 to -8
Correct on second attempt	+6

Question difficulty bands:

- 800–1000: Foundations (basic loops, arrays, string manipulation)
- 1000–1200: Core DSA (stacks, queues, linked lists, basic trees)
- 1200–1400: Intermediate (BFS, DFS, binary search, recursion, hashmaps)
- 1400–1600: Advanced (dynamic programming, graphs, backtracking)
- 1600–1800: Expert (system design components, complex DP, segment trees)
- 1800+: FAANG-level (hard LeetCode equivalents, optimization problems)

4.2 Knowledge Gap Detector

The AI tracks every failure at the sub-topic level — not just "Arrays" but "Two-pointer technique", "Sliding window", "Array rotation". If a user fails 3 or more questions in a sub-topic, the system flags it as a weak zone.

- Weak zones shown as red on the topic mastery map
- System automatically routes 2 easier foundational questions on the weak topic before advancing

- Notification: "You have been struggling with Dynamic Programming base cases. Here is a targeted warm-up."
- Weak zones cleared when user solves 3 consecutive questions in that sub-topic without hints

4.3 Session Configurator

Before every session, the user sees a setup screen. They choose the rules of the session. This is what transforms iteratr from a generic quiz platform into a personalized tool.

- Topic picker: Arrays, Trees, Graphs, DP, OS Concepts, Networking, System Design, Mixed
- Question format: MCQ only, Fill in the blank, Code space, Mixed
- Difficulty mode: Auto (Elo-matched), Easy warm-up, Hard grind, Interview prep
- Language: Python, C++, JavaScript
- Time pressure: None, Soft (timer shows), Hard (timer affects Elo)
- Interviewer style: Friendly mentor, Neutral, Strict FAANG interviewer
- Custom prompt box: free-text field — "I have a Google interview in 2 days, no hints allowed, be brutally honest"

4.4 Structured Output Generation (Backend)

The AI never generates free-text questions. Every question is forced into a strict Pydantic / Zod schema. This guarantees consistent formatting, correct test cases, and accurate Elo tagging. Malformed questions can never reach the user.

Question Schema (simplified)

```
type: 'mcq' | 'fill' | 'code' | 'order' | topic: string | subtopic: string | difficulty_elo: number |
problem_statement: string | payload: MCQPayload | FillPayload | CodePayload | OrderPayload |
hints: string[4] | explanation: string | tags: string[]
```

5. The Socratic Hint Engine (Core Differentiator)

This is the feature that makes iteratr different from every other platform. When a user gets an answer wrong, iteratr never says 'Incorrect. The answer is X.' Instead, the AI reasons about WHY the user was wrong and delivers a precisely calibrated hint.

Design principle: A hint should close the gap between what the user knows and what they need to know — without bypassing the thinking.

5.1 Hint Levels

Level	What the hint does
Level 1 — Concept gap	Explains WHY the submitted answer is logically wrong. Does not point toward the correct answer. Example: 'O(log n) is for binary search trees where you halve the search space. A hash map does not search — it computes.'
Level 2 — Directional nudge	Points toward the general concept without naming it. Example: 'Think about what operation a hash map performs that makes searching unnecessary.'
Level 3 — Structural scaffold	For code questions: gives pseudocode structure without working syntax. Example: 'You need: a pointer for prev (starts None), a pointer for curr, and a way to save next before you overwrite it.'
Level 4 — Full walkthrough	Complete explanation with annotated code. Awarded only after 3 failed attempts. Also flags the topic as a weak zone in the user's profile and applies a small Elo penalty.

5.2 The ReAct Agent Loop (Code Questions)

For code space questions, the hint is not just looked up from a database. A ReAct (Reason + Act) agent actively analyzes the failure:

1. User submits code.
2. Agent Reason: 'I need to run this against hidden test cases.'
3. Agent Action: Calls ExecuteCodeTool — sends code to Judge0 sandbox.
4. Observation: Code failed on test case 3 (empty array input).
5. Agent Reason: 'User missed the empty array edge case. They should not be told directly. I will hint about edge case handling.'
6. Agent Output: Level 1 hint targeting the specific gap identified.

This means two users who write different wrong code get different hints — because the agent reasons about their specific failure, not a generic template.

6. Phase 3 — Mock Interview Suite

Goal: Make iteratr the place developers go the week before their interview.

6.1 AI Interview Panel

A real-time mock interview experience powered by two AI agents running in parallel via WebSocket connections.

Agent	Role
Agent 1 — The Interviewer	Conducts the interview in real time. Asks DSA or system design questions, responds to the user's answers, asks follow-up questions, simulates interruptions and clarifying questions. Fully conversational.
Agent 2 — The Silent Grader	Runs invisibly in the background. Monitors the user's code quality, response latency, communication clarity, and problem-solving approach. Writes a score to the database at the end of the session. Never speaks during the interview.

6.2 Post-Interview Scorecard

After every mock interview, the user receives a detailed scorecard. This is the output they can share, save, and use to track improvement over time.

- Code quality score (0–100): correctness, time complexity awareness, edge case handling
- Communication score (0–100): clarity of explanation, asks good clarifying questions, thinks out loud
- Speed score (0–100): time to first approach, time to working solution
- Hint dependency score: how many hints were needed, at which stages
- Full transcript: every message and code submission with AI commentary
- "What you should have said/coded differently": specific, actionable feedback on each section
- Overall interview grade: Pass / Strong Pass / No Hire / Strong No Hire

6.3 Company Mode

Users can select a company profile before starting a mock interview. The AI Interviewer adapts its style, question types, and difficulty to match that company's known interview pattern.

- FAANG mode: heavy DSA focus, strict time limits, optimal solution expected, no hints given
- Startup mode: practical problem-solving, system design emphasis, communication weighted higher
- Product company mode: balanced DSA and product thinking, behavioral questions mixed in
- Custom mode: user writes their own company description and interview style

Viral growth mechanic

After a Company Mode interview, the user gets a shareable card: 'I just scored 84/100 on iteratr FAANG mode. Communication: 91, Code Quality: 78.' This is designed to be posted on LinkedIn and Twitter before real interviews.

6.4 Session Replay

Every interview session is recorded and stored as a full transcript with AI annotations. Users can review any session and see exactly where they went wrong and what a stronger answer would have looked like.

- Full message-by-message transcript
- Code submissions shown with syntax highlighting
- AI commentary on each section: 'At this point, you should have asked about the constraints.'
- Side-by-side comparison: user's code vs optimal solution
- Shareable replay link (optional, user controls privacy)

7. Phase 4 — Analytics & Growth

Goal: Make progress visible. Give users a reason to come back every day and a reason to share.

7.1 Skill Heatmap

A GitHub-style activity grid combined with a topic coverage map. The heatmap shows daily activity. The topic map shows which subjects are strong (green), weak (red), or untouched (gray). This is the dashboard users will screenshot and share.

- Activity grid: 52-week view of daily question attempts
- Streak counter: current streak and longest streak prominently displayed
- Topic coverage radar: all major DSA topics shown with fill level (0-100%)
- Elo history line chart: rating over the past 30 days with milestone markers
- Weak zone callouts: top 3 areas to focus on this week

7.2 Daily Challenge & Streak System

One new question every day, matched to the user's current Elo. Solving it extends the streak. Missing a day does not break the streak — but it pauses the daily bonus multiplier.

- Daily challenge unlocks at midnight local time
- Streak bonus: consecutive days multiply Elo gains by 1.1x, 1.2x, up to 1.5x
- Grace period: one streak freeze per week if the user misses a day
- Weekly challenge: harder question released every Monday, bonus Elo on completion

7.3 Shareable Public Profile

Every user gets a public profile page at [iteratr.online/u/\[username\]](https://iteratr.online/u/[username]). This page is designed to be put on a resume or LinkedIn profile.

- Elo rating displayed prominently with percentile (e.g. Top 12%)
- Topics mastered: list of sub-topics where the user has solved 5+ questions without hints
- Mock interview best score per company mode
- Activity streak and total questions solved
- "Interview Ready" badge unlocked after scoring 80+ on a FAANG mode mock interview
- User controls exactly which sections are public vs private

7.4 Long-Term Memory & Reflection

The platform builds a behavioral model of each user over time. This is stored as a structured reflection in the database and injected into the AI's system prompt on every new session.

- After each session: AI writes a 2-sentence reflection on what the user struggled with
- Example: 'This user consistently misses edge cases on null inputs. Use a real-world analogy before hinting at pointer logic.'
- Reflection is updated after every 5 sessions
- User can view and reset their reflection at any time
- This is what makes iteratr feel like it knows you — not just a stateless quiz machine

8. Technology Stack

8.1 Full Stack Overview

Layer	Technology & Reason
Frontend	Next.js 14 (App Router) + React. Fast, server-side rendering, great developer experience. Deployed on Vercel.
Code editor	Monaco Editor. Powers VS Code. Syntax highlighting, autocomplete, and fold regions out of the box.
Backend / API	Node.js with Next.js API routes for simple endpoints. Dedicated Express/Node service for heavy agent logic.
Database	PostgreSQL via Supabase. Handles users, questions, attempts, Elo history, session transcripts.
Vector database	pgvector extension on PostgreSQL. Stores embeddings of documentation for RAG-based hints.
Code execution	Judge0 API (Phase 1–2). Self-hosted Piston (Phase 3+). Secure Docker sandboxing.
AI framework	LangChain.js for agent orchestration and tool calling. Structured outputs via Zod schemas.
Core LLM	Google Gemini 1.5 Pro (primary) or OpenAI GPT-4o (fallback). Used for question generation and hint generation.
Real-time	WebSockets via Socket.io. Used for mock interview streaming in Phase 3.
Caching / queues	Redis (Upstash). Session state, rate limiting, and WebSocket room management.
Authentication	NextAuth.js with Google and GitHub OAuth providers.
Deployment	Vercel (frontend + serverless API). Railway or Fly.io (long-running agent service).

8.2 Database Schema (Core Tables)

Table	Key fields
users	id, email, display_name, elo_rating, streak_count, reflection_text, created_at
questions	id, type, topic, subtopic, difficulty_elo, problem_statement, payload (JSONB), hints (array), tags (array)

attempts	id, user_id, question_id, submitted_answer, is_correct, hints_used, time_taken_seconds, elo_change, created_at
sessions	id, user_id, session_type, config (JSONB), score_code, score_comms, transcript (JSONB), created_at
elo_history	id, user_id, elo_before, elo_after, reason, created_at
topic_stats	id, user_id, topic, subtopic, solved_count, fail_count, is_weak_zone

9. Build Roadmap

The roadmap is structured to ship a working, valuable product as early as possible. Do not wait for Phase 4 to launch. The MVP after Phase 2 is already better than every existing platform.

Phase	Milestone	Duration	Ship condition
1A	Next.js + Monaco Editor + OAuth login	Week 1–2	User can log in and see a code editor
1B	Judge0 integration + basic MCQ questions	Week 2–3	User can run Python code and answer MCQ
1C	Fill in blank + Drag to order + hint engine	Week 3–4	All 4 question types working with basic hints
2A	PostgreSQL schema + Elo calculation	Week 5–6	Elo updates after every attempt, stored correctly
2B	Session configurator + custom prompt injection	Week 6–7	User can configure a session before starting
2C	Knowledge gap detector + topic stats	Week 7–8	Weak zones detected, easier questions routed
MVP LAUNCH	Public beta at iteratr.online	Week 8	Share with friends, get first 100 users
3A	WebSocket setup + Agent 1 (Interviewer)	Week 9–11	Live chat interview works end to end
3B	Agent 2 (Silent Grader) + Scorecard	Week 11–13	Full mock interview with post-session report
4A	Activity heatmap + Elo history chart	Week 14–15	Dashboard with full analytics
4B	Public profiles + streak system	Week 15–16	Shareable profiles live, daily challenge active

10. Monetization Strategy

iteratr uses a freemium model. The free tier is generous enough to create genuine value and word-of-mouth growth. The Pro tier unlocks the features that serious job-seekers need.

Feature	Free	Pro (\$12/mo)
Daily questions	5 per day	Unlimited
Question types	MCQ + Fill only	All 4 types including Code Space
Code execution	Not included	Full Judge0 execution
Hint engine	Level 1 only	All 4 levels
Mock interviews	1 per month	Unlimited
Company mode	Not included	All company modes
Session replay	Not included	Full transcript + AI commentary
Shareable profile	Basic stats only	Full profile with Interview Ready badge
Custom prompt	Not included	Full custom prompt injection
Long-term memory	Not included	Full reflection and personalization

Revenue targets

100 free users by Week 10 → 10 Pro conversions (10% conversion) → \$120 MRR. Target 1000 free users by Month 4 → 80 Pro conversions → \$960 MRR. At 5000 users → \$4,800 MRR covers infrastructure and pays one developer salary.

11. Competitive Positioning

Platform	Why iteratr wins
LeetCode	LeetCode shows you the answer after you fail. iteratr never shows the answer — it teaches. iteratr also has adaptive difficulty; LeetCode is static Easy/Medium/Hard.
HackerRank	HackerRank is used by companies for testing, not learning. It has no hint engine, no Elo system, and no personalization. It feels like an exam, not a mentor.
Codewars	Codewars is community-driven — solutions vary in quality and the feedback is whatever another user wrote. iteratr feedback is personalized to YOUR specific failure.
ChatGPT / Claude	General AI gives you the answer immediately if you ask for it. iteratr is specifically designed to NOT give the answer — it guides. It also has structured question delivery, code execution, Elo, and analytics that a chatbot does not have.

iteratr

Code. Fail. Understand why. Improve.

iteratr.online